

Kapitel 15

Probleme der Rechnerarithmetik

15.1 Gleitkomma–Ausnahmebehandlung

Numerisches Rechnen mit Computern ist nicht unproblematisch. Das gilt besonders für das Rechnen mit Gleitkommawerten, in gewissem Maße aber auch für das Rechnen mit ganzzahligen Werten. Diese Probleme sind keineswegs C–spezifisch, sondern treten bei allen Sprachen auf, die die entsprechenden Datentypen kennen, und haben eine einzige Ursache, nämlich die begrenzte Stellenzahl, die dargestellt werden kann.

Erinnern Sie sich noch einmal an die Minimalanforderungen für `double`–Zahlen, die am Anfang des Kurses genannt wurden: Der geforderte minimale Wertebereich ist

$$[-10^{+37}, -10^{-37}] \cup \{0\} \cup [+10^{-37}, +10^{+37}]$$

bei einer minimalen Dichte von 10^{-9} . Hieraus ergeben sich direkt die drei „problematischen“ Fälle:

- Es kann ein *Überlauf* (*overflow*) eintreten, d.h. ein Wert wird dem Betrage nach größer als `DBL_MAX`.
- Es kann ein *Unterlauf* (*underflow*) eintreten, d.h. ein Wert wird dem Betrage nach kleiner als `DBL_MIN`.
- Ein Wert fällt in eine „Lücke“ zwischen zwei darstellbaren Zahlen, d.h. er muss gerundet werden.

Wie auf solche Fälle reagiert wird, kann von Rechner zu Rechner sehr unterschiedlich sein. Man kann deshalb nur „übliche“ Reaktionen beschreiben; andere Reaktionen sind denkbar.

- Beim Überlauf gibt es drei „übliche“ Reaktionen:
 - Das Programm wird abgebrochen.
 - Der Wert wird auf „Infinity“ gesetzt, einen ausgezeichneten Wert.
 - Der Wert wird undefiniert.

Division durch Null wird mit gewisser Berechtigung in der Regel wie ein Überlauf behandelt.

- Bei Unterlauf ist es üblich, den Wert durch Null zu ersetzen, ohne dass dieses dem Programm angezeigt wird.
- Gerundet wird in der Regel automatisch, ohne dass dieses dem Programm angezeigt wird.

Nur für den Überlauf schreibt C vor, dass ein Programm selbst festlegen kann, wie darauf zu reagieren ist. Die entsprechenden Mittel stellt C mit der Headerdatei `signal.h` zur Verfügung.

15.2 Überlauf bei ganzen Zahlen

Die ganzzahligen Typen sind auf der einen Seite weniger problematisch, da bei ihnen nur der Überlauf eintreten kann, und auf der anderen Seite problematischer, da die Verhinderung/Vermeidung von Überläufen grundsätzlich und ausschließlich in die Verantwortung des Programmierers gestellt ist.

Überlauf bei ganzzahligen Operationen bewirkt nämlich in der Regel nichts – außer dass mit falschen Werten weitergerechnet wird. Eine Ausnahme sind Divisionen durch Null und natürlich Berechnungen des Restes mit Divisor Null, die vielfach einen Programmabbruch bewirken.

Der Hintergrund für die üblicherweise verschiedenen Vorgehensweisen sind Hardware-Eigenschaften der Rechner: Bei Gleitkommaoperationen ist jeder Überlauf und Unterlauf ein Fehler, der ohnehin besonders behandelt werden muss. Bei ganzzahligen Operationen tritt dagegen häufiger ein Hardware-Überlauf auf, *ohne* dass das ein Fehler ist. Wir sehen uns das an einem Beispiel an. Der Einfachheit halber wird Byte-Arithmetik betrachtet, auch wenn C sie nicht explizit vorsieht. Wir haben etwa

$$\begin{array}{r} 1111\ 1111_b \\ + \quad 0000\ 0001_b \\ \hline = 1\ 0000\ 0000_b \end{array}$$

wobei das werthöchste, neunte Bit gerade das überlaufende Bit ist, das nicht mehr gespeichert werden kann. Ob dieser Überlauf ein Fehler ist oder nicht, hängt davon ab, ob wir die Operanden als vorzeichenbehaftet oder vorzeichenlos betrachten:

- Bei vorzeichenbehafteter Operation haben wir gerade $-1 + 1 = 0$, also das korrekte Resultat!
- Bei vorzeichenloser Operation haben wir $255 + 1 = 0$ – und das ist offenbar falsch!

Sieht man sich das ganze genauer an, so stellt man fest, dass bei vorzeichenlosen Operationen modulo $\dots_MAX + 1$ gerechnet wird. Bei vorzeichenbehafteten Operationen kommt man zu einem ähnliche Ergebnis.

Übrigens: Bei den impliziten Typumwandlungen im Zuge von arithmetischen Operationen sind die Regeln gerade so angelegt, dass Überlauf und Unterlauf nicht auftreten können. Werden Typumwandlungen dagegen durch Wertzuweisungen oder Castoperatoren erzwungen, können Überlauf und Unterlauf wie bei arithmetischen Operationen auftreten.

15.3 Normalisierte Gleitkomma-Darstellungen

Wir haben gesehen, dass man im Programm oder auch bei der Eingabe für Gleitkommawerte beliebig viele verschiedene Darstellungen verwenden kann. Der Wert 1.2 kann etwa als 1.2, 12e-1, 0.12e1, usw., dargestellt werden.

Im Rechner gibt es dagegen für jeden darstellbaren Gleitkommawert nur genau eine Darstellung, die als *normalisierte Darstellung* bezeichnet wird. Prinzipiell gibt es unterschiedliche normalisierte Darstellungen (beschrieben in einer Norm von IEEE), viele Rechner verwenden von diesen jedoch nur genau eine.

Der Anschaulichkeit halber wird die normalisierte Darstellungen für dezimale Rechenwerke beschrieben. Als Beispiel nehmen wir ein Rechenwerk, das mit fünf signifikanten Stellen

und einem einstelligen Exponententeil arbeitet. Eine normalisierte Darstellung setzt den (gedachten) Dezimalpunkt direkt links neben die werthöchste signifikante Ziffer und passt den Exponenten entsprechend an; in ihr hat der Wert 1.2 also die eindeutige Darstellung

$$0.12000 \cdot 10^1$$

Der Wertebereich dieses Rechenwerkes ist dann

$$[-0.99999 \cdot 10^{+9}, -0.10000 \cdot 10^{-9}] \cup \{0\} \cup [+0.10000 \cdot 10^{-9}, +0.99999 \cdot 10^{+9}]$$

wobei Null ($0.00000 \cdot 10^0$) besonders definiert werden muss, da die Darstellung nicht normalisiert ist.

Wenn man mit solchen Darstellungen rechnen will, muss man sie in die beiden Bestandteile zerlegen, die beiden Bestandteile getrennt verarbeiten und dann wieder zu einer normalisierten Darstellung zusammenfügen.

15.4 Assoziativ- und Kommutativgesetz

Ausdrücke werden Schritt für Schritt ausgewertet – und in jedem Schritt muss der berechnete Wert innerhalb des Wertebereichs des Typs liegen. Wir sehen uns das für den Ausdruck $\frac{ab}{cd}$ an, der in `float`-Arithmetik ausgewertet werden soll:

A	B	C	D	A*B/C/D	A/C*B/D
10^{30}	10^{30}	10^{30}	10^{30}	Überlauf	1
10^{30}	10^{-30}	10^{-30}	10^{30}	1	Überlauf
10^{-30}	10^{-30}	10^{-30}	10^{-30}	Unterlauf	1
10^{-30}	10^{30}	10^{30}	10^{-30}	1	Unterlauf

Obwohl das theoretische Resultat in allen Fällen 1 ist, können Überlauf und Unterlauf auftreten!

Assoziativ- und Kommutativgesetz sind also auf einem Rechner für Gleitkommazahlen nicht erfüllt, da die Voraussetzungen für beide Gesetze nicht erfüllt sind: Zur Verfügung hat man ja nicht die reellen Zahlen, sondern nur eine Teilmenge, deren Werte nicht dicht in den reellen Zahlen liegen!

Folgerung: Man sollte sich bei Gleitkommaausdrücken genau überlegen, in welcher Reihenfolge man sie ausgewertet haben möchte. Der Standard unterstützt das, im Gegensatz zum „alten“ C: Scheinbar redundante Klammern dürfen nicht mehr „wegoptimiert“ werden! War ein Compiler früher berechtigt, in einem Ausdruck wie

$$a + (b + c)$$

die Klammern schlichtweg zu ignorieren (was unter Umständen zu „Klimmzügen“ im Programm zwang), so *muss* ein Standardcompiler sie jetzt berücksichtigen!

Beispiel: Sind $a = 1$, $b = -1e12$ und $c = 1e12$, dann ergibt sich bei einem Rechenwerk mit 10 signifikanten Stellen

$$\begin{aligned} a + (b + c) &= 1 \\ (a + b) + c &= 0 \end{aligned}$$

15.5 Rundungsfehler

Damit kommen wir auch schon zum nächsten Problem, nämlich der Rundung, die aufgrund der begrenzten Stellenzahl in vielen Fällen unumgänglich ist, und den sich daraus ergebenden *Rundungsfehler*.

Nehmen wir einmal an, wir hätten ein dezimales Rechenwerk zur Verfügung, das mit zwei Dezimalstellen Genauigkeit arbeitet. Dann lässt sich in diesem Rechenwerk die Zahl 1.5 exakt speichern. Bildet man jetzt jedoch das Produkt $1.5 * 1.5$, so resultiert die Zahl 2.25 mit drei signifikanten Stellen, die nicht mehr exakt gespeichert werden kann, sondern zunächst durch Rundung auf zwei signifikante Stellen verkürzt werden muss. In unserem Rechenwerk ist das Resultat, je nach Art der Rundung, also entweder 2.2 oder 2.3.

Genau diese Probleme treten bei jedem Rechner auf. Allerdings lässt sich das, was bei einem Rechner passiert, für einen Menschen nicht mehr so auf den ersten Blick überschauen: Der Mensch ist in seinen Zahlenvorstellungen sehr stark auf das Dezimalsystem fixiert, während die Rechner ja mit binären Zahlendarstellungen arbeiten.

Und so wird vielfach übersehen, dass Rundungsfehler nicht erst beim eigentlichen Rechnen, sondern bereits bei der Umwandlung der dezimalen Konstantendarstellung in die interne Darstellung auftreten. Dazu ein Beispiel: Der „schöne“ Dezimalbruch 0.2_d ergibt bei Umwandlung in Binärdarstellung den periodischen Bruch $0.001100110011\dots_b$, lässt sich also im Rechner nur gerundet darstellen.

Bei arithmetischen Operationen mit gerundeten Zahlen können die Rundungsfehler unter Umständen erheblich anwachsen. Dazu das folgende Beispiel, das 10000 Mal 0.2 addiert und das Ergebnis ausgibt:

```
float schritt = 0.2f, summe = 0.0f;
int i;

for (i = 0; i < 10000; i++)
    summe += schritt;

printf("%f\n", summe);
printf("%f\n", 10000 * schritt);
```

Ein Testlauf ergab für die erste Ausgabe den Wert 1999.805835 und für die zweite Ausgabe 2000.000000.

Der Programmierer muss sich also sehr genau überlegen, wie er sein Programm formuliert, um den Einfluss der Rundungsfehler möglichst gering zu halten. Dazu ein Beispiel: Eine Funktion $f(x)$ sei in einem Intervall $[a, b]$ zu tabellieren, wobei die Anzahl N der Teilintervalle vorgegeben ist.

Als „naive“ Lösung bietet sich an, zunächst einmal die Schrittweite zu berechnen und ihren Wert dann bei jedem Schleifendurchlauf zur Bestimmung des nächsten Tabellierungspunktes zu verwenden. Das Programm hat dann das folgende Schema:

```
h = (b - a) / N;
x = a;
while (x <= b)
{
    printf("%f %f\n", x, f(x));
    x += h;
}
```

Mit den Werten $a = 0$, $b = 4$ und $N = 40$ erhält hier die Variable x vor dem letzten Schleifendurchlauf den Wert 3.9999980, erreicht also nicht exakt den rechten Rand des Ta-

bellierungsintervalls. Entsprechend ungenau werden natürlich auch die tabellierten Funktionswerte.

Günstiger ist es, nicht mit der Schrittweite zu arbeiten, sondern die Tabellierungspunkte jeweils direkt zu berechnen, etwa nach dem folgenden Schema

```
i = 0;
while (i <= N)
{
    x = ((N - i) * a + i * b) / N
    printf("%f %f\n", x, f(x));
    i++;
}
```

Hier wird, wieder mit den Werten $a = 0$, $b = 4$ und $N = 40$, beim letzten Schleifendurchlauf der rechte Rand des Tabellierungsintervalls exakt erreicht.

Auch bei Typumwandlungen können Rundungsfehler auftreten – und das nicht nur bei erzwungenen Umwandlungen: Für den Typ `float` sind nur 6 signifikante Stellen vorgegeben, während der Typ `long` über mindestens 10 Stellen verfügen muss. Bei einer Umwandlung von `long` in `float`, wie sie ggf. automatisch bei der Auswertung von Ausdrücken erfolgt, können also durchaus die letzten Stellen des `long`-Wertes verloren gehen.

15.6 Auslöschung

Wir haben eben gesehen, dass sich die Rundungsfehler bei Gleitkomma-Arithmetik in bestimmten Situationen beherrschen oder doch zumindest reduzieren lassen. In anderen Situationen ist das nicht so einfach oder sogar unmöglich.

Wir betrachten zunächst wieder ein Beispiel: Gegeben seien Zahlen

$$\begin{aligned}x_1 &= 0.42178 \cdot 10^2 \\x_2 &= 0.23722 \cdot 10^3 \\y_1 &= 0.99986 \cdot 10^2 \\y_2 &= 0.99987 \cdot 10^2\end{aligned}$$

Dann gilt

$$\begin{aligned}x_1 x_2 &= 0.1000546516 \cdot 10^5 \\y_1 y_2 &= 0.9997300182 \cdot 10^4\end{aligned}$$

und

$$\begin{aligned}x_1 x_2 - y_1 y_2 &= 0.10005465160 \cdot 10^5 \\&\quad - 0.09997300182 \cdot 10^5 \\&\quad \hline &= 0.00008164978 \cdot 10^5 \\ \text{normalisiert:} &\quad 0.81649780000 \cdot 10^1\end{aligned}$$

Das zwischenzeitliche Vorkommen führender Nullen wird als *Auslöschung* bezeichnet.

Erlaubt die verfügbare Arithmetik nicht die exakte Darstellung der Zwischenresultate, sondern z.B. nur fünfstellige Resultate, so werden bereits die Produkte $x_1 x_2$ und $y_1 y_2$ gerundet geliefert:

$$x_1 x_2 \approx x = 0.10005 \cdot 10^5 \quad \text{und} \quad y_1 y_2 \approx y = 0.99973 \cdot 10^4$$

Erst danach kann die Subtraktion ausgeführt werden und liefert

$$\begin{array}{rcl}
 x - y & = & 0.100050 \cdot 10^5 \\
 & - & 0.099973 \cdot 10^5 \\
 \hline
 & = & 0.000077 \cdot 10^5 \\
 \text{normalisiert:} & & 0.770000 \cdot 10^1 \\
 \text{gerundet:} & & 0.77000 \cdot 10^1
 \end{array}$$

Der Vergleich der Differenzen $x_1x_2 - y_1y_2$ und $x - y$ zeigt, dass das gerundete Resultat zwar dieselbe Größenordnung besitzt wie das exakte Resultat, dass aber nicht einmal die werthöchsten Ziffern übereinstimmen.

Auslöschung tritt immer dann ein, wenn zwei beinahe gleichgroße Zahlen subtrahiert werden. Selbst wenn die einzelne Subtraktion fehlerfrei ausgeführt wird, ist die Auslöschung numerisch gefährlich, wenn die zu subtrahierenden Daten selber bereits gerundet wurden.

Im Beispiel, das wir eben betrachtet haben, hatte das tatsächliche Resultat noch die Größenordnung des theoretischen Resultats. Es bereitet jedoch keine Probleme, Beispiele zu finden, in denen auch die Größenordnungen voneinander abweichen. Dazu betrachten wir die Taylorreihe der Exponentialfunktion

$$e^z = \sum_{k=0}^{\infty} \frac{z^k}{k!}$$

Diese Reihe ist absolut konvergent für jedes z aus der komplexen Zahlenebene. Für reelles, negatives z haben die Summanden alternierende Vorzeichen. In diesem Fall ist der Fehler, den man begeht, wenn man die n -te Partialsumme als Wert für e^z nimmt, kleiner als der $(n+1)$ -te Summand, der als erster vernachlässigt wird.

Wertet man diese Reihe unter Verwendung einer Arithmetik mit 6-stelliger dezimaler Mantisse für $z = -14$ aus, so erhält man nach 50 Additionen den Wert -0.159691 – das korrekte Resultat wäre $0.00000083152900!$ (Der Abbruch der Summation nach 50 Schritten erfolgt, weil alle weiteren Summanden so klein sind, dass sie bei der verwendeten Arithmetik keinen Beitrag mehr zu der Summe liefern.)

Hier lässt sich mit Mitteln der Programmieretechnik nichts mehr ausrichten. Abhilfe schaffen nur noch besondere Algorithmen, die die Probleme der Gleitkommaarithmetik berücksichtigen, oder der Einsatz der Intervallrechnung.